

PHOTO PROCESSING AND INDESIGN AUTOMATION PIPELINE FOR YEARBOOK PHOTOGRAPHY AT LTID JAPAN

Vaibhav Chamoli (Roll Number: 23035010039)

Internship Report — Trimester 7

B.Sc. (Honours) Data Science and Artificial Intelligence
Indian Institute of Technology Guwahati, India
c.vaibhav@op.iitg.ac.in

ABSTRACT

This report presents work carried out during an ongoing six-month internship at Long Term Industrial Development (LTID), a Japanese yearbook photography company, on the problem of automating portrait selection and page layout for school yearbooks. Manually processing 200–400 students per school takes several hours of skilled labour and is prone to errors, especially when the shooting order is disturbed. To address this, a four-stage pipeline was developed that automatically identifies students using face recognition (Insight-Face), reads placard cards using optical character recognition (EasyOCR), selects the best portrait from multiple candidates using a quality scoring model, computes face-aligned placement offsets using MediaPipe Face Mesh, and populates Adobe InDesign templates through two custom automation scripts. The key design idea — clustering photos by facial identity *before* reading attendance numbers — makes the system robust to out-of-order shooting. The pipeline was tested on approximately 3,000 students across multiple schools. Results show an automatic identification rate of 97–98% and a face-alignment success rate of 86.9% on a 99-entry reference dataset, with zero complete failures.

Index Terms— Face Recognition, ArcFace, RetinaFace, MediaPipe Face Mesh, EasyOCR, Yearbook Photography, InDesign Automation

1. INTRODUCTION

Japanese school yearbook production requires photographing entire graduating classes — typically 200 to 400 students per school — and placing each portrait at a precise, chin-anchored position inside a strictly templated Adobe InDesign spread. Before this project, the workflow at LTID was entirely manual. An operator would look at each photo, read the number on the placard card the student held, pick the best portrait out of five to eight candidates, rename the file to match a fixed naming convention, and drag it into the correct InDesign frame. A single school could take several hours, and mislabelling was common whenever the shooting order got disrupted — a student turning to chat with a friend, an operator handing the camera to someone else mid-session, or a placard being partially blocked by the student’s hand.

I joined LTID in late November 2025 for what is now a six-month internship, still ongoing with regular client check-ins. The company is Japanese, so all communication with the client goes through Yasuo Tanguchi and Natalia-san, who translate between our team and the LTID photographers. This added a layer of iteration to

the feedback loop — I could not simply ask the client a quick question, so each design decision had to be carefully thought through before presenting it.

My specific contributions to the project include the face-embedding-first design concept (which I proposed at the very start and which has remained the backbone of the pipeline through every subsequent iteration), the face clustering and grouping module (Stage 2), the package exporter (Stage 3), the MediaPipe-based face alignment module (Stage 4), and both companion Adobe InDesign JSX automation scripts. The roster parser and some OCR refinements were handled by other team members during collaborative work. The OCR backend started as PaddleOCR [1] — my initial choice because of its strong Japanese recognition — but was later migrated to EasyOCR [2] to resolve runtime dependency conflicts, which I discuss in detail in Section 3.

The rest of this report is structured as follows. Section 2 lays out the problem and objectives. Section 3 walks through the pipeline design and key technical decisions. Section 4 presents results from pilot runs on real school data. Section 5 discusses trade-offs and limitations honestly. Section 6 wraps up with remaining work.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1. Problem Statement

The input to the system is (i) a flat folder of roughly 800 to 1,600 JPEG images from a single shooting session and (ii) a school roster in Excel `.xlsx` or scanned PDF form, listing each student’s attendance number ($n \in [1, 50]$), kanji name, and furigana reading. The system must produce:

- a per-class folder of correctly named files following the convention `[YY]-[School]-[Original]-[GCN]-[Tag].ext`,
- a single best-shot portrait per student,
- a `manifest.json` with per-student face-offset parameters (`offsetX`, `offsetY`, `scaleFactor`) in millimetres,
- a fully populated InDesign document where every face sits at the same chin position ($t_{\text{chin}} \approx 1.06$ frame-height units).

On top of this, the system has to deal with Japanese-text teacher cards (kanji name, class label, role), absent students, duplicate placard reads, extraction of student data from scanned PDF rosters using vision-based OCR (many Japanese schools hand over scans rather than spreadsheets), and the wide variation in lighting, pose, and expression that naturally occurs across hundreds of consecutive photographs.

2.2. Objectives

1. Replace the order-dependent matching approach with face-embedding-based clustering using deep face representations [3, 4].
2. Build robust placard OCR for digit cards and Japanese teacher cards, with a deterministic fallback path.
3. Score portrait quality with a multi-signal model combining facial landmarks, head pose, eye openness [5], sharpness [6], and brightness, falling back to a Viola–Jones [7] cascade when InsightFace fails.
4. Compute per-student InDesign placement offsets using MediaPipe Face Mesh [8].
5. Automate InDesign layout end-to-end through ExtendScript, removing all manual photo placement.

3. METHODOLOGY

3.1. System Architecture

The pipeline runs in Google Colab on a T4 GPU and has four stages (Table 1). Every Python module shares the same dataclass hierarchy (ScoredPhoto, StudentPhotoGroup, TeacherPhotoGroup, ClassPhotoGroup), so each stage feeds directly into the next.

Table 1. Pipeline stages and their outputs

Stage	Output
1. Roster parsing	<code>roster.json</code>
2. Face grouping + OCR	per-class <code>ClassPhotoGroup</code>
3. Package export	named files + <code>manifest.json</code>
4. Face alignment	manifest with face offsets

3.2. Roster Parsing (Stage 1)

The roster parser (a team-contributed module) turns a school’s student roster into a normalised `roster.json` that every downstream stage relies on. It auto-detects three different Excel layouts:

- **Format A** (wide HR-block): all classes sit as column blocks on one sheet, with a configurable column stride read from a sheet. This is the format Chihaya High School uses.
- **Format B** (paired-class sheets): each worksheet holds one or two classes with a header row for teacher name, class label, and student count.
- **Generic fallback**: scans every sheet looking for columns labelled `No/` and `,` picking up optional furigana and gender columns along the way.

When the school provides a scanned PDF instead of a spreadsheet (which happens more often than one might expect), the parser renders each page to an image with PyMuPDF and sends it to GPT-4o vision [9] with a structured prompt. The model extracts class labels, teacher names, and student records as JSON. Multi-page PDFs get processed page by page and merged, with duplicate detection across pages. The parser also validates the output — checking for gaps or duplicates in attendance numbers — and flags anything suspicious for manual review.

Two things from this stage feed into later stages: the set of valid attendance numbers per class (used to filter OCR candidates in Stage 2), and the student names and furigana (written directly into the InDesign manifest for name placement).

3.3. Face-Embedding-First Grouping (Stage 2)

This is the core of the pipeline and the design decision I am most invested in. I proposed this approach at the very start of the project, and it has survived every round of iteration since.

The idea is simple but important: cluster photos by *who is in them* before trying to figure out *what number they hold*. The old way of doing things — look at a card, then grab the next few portraits — breaks the moment shooting order is disturbed. And in practice, order gets disturbed constantly: a student steps out of line, a photographer re-takes a placard shot later, or the person running the session swaps with someone else.

The module uses InsightFace’s `buffalo_l` pack, which bundles RetinaFace for detection (WIDER FACE hard-set AP of 91.4%, easy-set 96.9% [4]) and an ArcFace-trained backbone for recognition (LFW 99.83%, CFP-FP 98.37%, AgeDB 98.15% [3]). Each detected face gets turned into an ℓ_2 -normalised 512-dimensional embedding $\mathbf{e}$. For every image that is not a class separator, the largest face is matched against a running set of cluster centroids $\{\mathbf{c}_k\}_{k=1}^K$ using cosine similarity:

$$\text{sim}(\mathbf{e}, \mathbf{c}_k) = \mathbf{e}^\top \mathbf{c}_k, \quad \|\mathbf{e}\| = \|\mathbf{c}_k\| = 1. \quad (1)$$

If the best match $s^* = \max_k \text{sim}(\mathbf{e}, \mathbf{c}_k)$ exceeds $\tau = 0.55$, the embedding joins that cluster and the centroid updates as a running average; otherwise a new cluster is created:

$$\text{assign} \rightarrow \begin{cases} k^*, & s^* \geq 0.55 \\ \text{new cluster}, & s^* < 0.55 \end{cases} \quad (2)$$

Choosing $\tau = 0.55$ took some experimentation. At $\tau = 0.65$, classes with 30-odd students were getting split into too many clusters because slightly different angles or expressions pushed similarity below the threshold. At $\tau = 0.45$, different students started merging into the same cluster. 0.55 was the sweet spot on our internal validation runs. Since each class only has around 30 to 40 students — well below the point where ArcFace embeddings start to crowd [3] — the centroid-based scheme works reliably without needing a more complex clustering algorithm.

Once clusters are formed, each one gets its attendance number by looking at the OCR results across all images in that cluster. Class membership is figured out by checking where the median image index of a cluster falls relative to the class separator cards (the “3-1” cards that mark class boundaries).

3.4. Card Detection and OCR (within Stage 2)

The pipeline has to tell apart three kinds of cards: class separators (“3-1” style labels), Japanese teacher cards (kanji name plus class and role), and student number cards (digits 1 through 50). The detector crops a candidate card region using HSV thresholding and contour analysis, with constraints on aspect ratio ($1.2 < w/h < 3.5$), vertical position ($y > 0.30H$ — cards are held at waist level, not near the face), and rectangular fill (≥ 0.65) [10].

One optimisation I made early on was to restrict OCR to the middle portion of the image rather than scanning the entire frame. This cut down processing time noticeably and also reduced false positives from background text.

I originally chose PaddleOCR [1] for its strong Japanese character recognition — it consistently outperforms other open-source OCR engines on Chinese and Japanese benchmarks [11]. However, PaddleOCR turned out to be heavy in terms of both runtime and GPU memory. More critically, its PaddlePaddle backend clashed with the

ONNX Runtime and CUDA dependencies that InsightFace needed, causing frequent crashes in Colab after runtime restarts. We eventually migrated to EasyOCR [2] (CRAFT detector + ResNet-CRNN with CTC), which is lighter and fits cleanly alongside InsightFace and MediaPipe in a single dependency tree. I kept the multi-variant contrast preprocessing from the PaddleOCR era, which helps with faded or low-contrast cards.

For digit cards, any OCR candidate with confidence above 0.50 (or 0.80 when no card contour was found) gets filtered first by the roster’s valid numbers, then by a 1–50 sanity range. The highest-confidence survivor wins. Teacher cards are trickier — they contain kanji names and role labels like (teacher), (homeroom teacher), or (deputy). I implemented four cascading name-extraction strategies: anchoring on and taking the text before it, checking the line above when sits in its own box, regex-based extraction, and falling back to the longest kanji-rich line (≥ 2 kanji, length 2–12).

3.5. Portrait Quality Scoring (within Stage 2)

When I first started working on the pipeline, the problem was framed differently. Initially, I only had a single image per student and the task was to *enhance* it — adjusting contrast, sharpness, and colour balance to make it look presentable. I spent a lot of time on image processing techniques and had to check with the client after every adjustment, going back and forth through Yasuo Tanguchi and Natalia-san. Eventually the workflow shifted: the photographers started providing multiple shots per student, and the problem became *selection* rather than enhancement. That pivot made the scoring model necessary.

Each portrait now receives a scalar score from five signals, weighted as:

$$\text{raw} = 0.10 d + 0.30 p + 0.30 e + 0.15 q + 0.15 b, \quad (3)$$

where d is the detection confidence from InsightFace; p is a head-pose score that penalises pitch beyond 18° , yaw past $\pm 25^\circ$, and roll past $\pm 20^\circ$, with the full penalty accumulated as:

$$\pi_p = \min\left(1, \frac{\max(0, \text{pitch} - 18)}{20} + 0.5 \cdot \frac{\max(0, -15 - \text{pitch})}{20} + \frac{\max(0, |\text{yaw}| - 25)}{25} + 0.6 \cdot \frac{\max(0, |\text{roll}| - 20)}{20}\right);$$

e measures eye openness using the Eye Aspect Ratio (EAR) from Soukupová and Čech [5]:

$$\text{EAR} = \frac{\|p_2 - p_6\| + \|p_3 - p_5\|}{2 \|p_1 - p_4\|}, \quad (4)$$

with thresholds I calibrated from our actual portrait data — eyes are considered open at $\text{EAR} \geq 0.30$ and closed at ≤ 0.16 ; q is sharpness via Laplacian variance [6], maxing out at $\sigma^2 \geq 200$; and b penalises brightness that drifts from a target grey level of 140.

On top of the raw score, a gesture penalty catches students doing peace signs or pointing at the camera. It works by building an HSV skin-colour mask, zeroing out the face and neck area, and checking the upper 70% of the frame for skin blobs. If more than 2% of that region is exposed skin (outside the face), the penalty kicks in; at 8% it saturates. Even small isolated blobs ($\geq 0.3\%$ of the region) trigger a partial penalty, which catches a single raised fist. The penalty multiplier is $g_m = 1 - 0.80g$.

When InsightFace cannot detect a face at all (which happens occasionally on extreme profile shots), the system falls back to a Viola–Jones Haar frontal-cascade [7] with a simpler scoring formula. An

optional GPT-4o vision backend [9] was also implemented for comparison, but the local scorer remains the default in production to avoid API costs.

3.6. Face-Aligned Placement for InDesign (Stage 4)

Face alignment was one of the problems that evolved most during the internship. When the requirement first came in, I hardcoded the offsets — I looked at a batch of images, eyeballed the right *offsetX*, *offsetY*, and *scaleFactor* values, and used those fixed numbers for everyone. It worked reasonably well when all students had similar face sizes and hair heights, but it fell apart on students with very different proportions. That is when I wrote the offset calculator.

The module uses MediaPipe Face Mesh [8], which estimates 468 3D landmarks from a single camera input in real time. I only need three of them: chin (vertex 152), left eye (vertex 33), and right eye (vertex 263). From these, plus the InDesign frame dimensions read from `frame_config.json`, the module simulates InDesign’s `FILL_PROPORTIONALLY` fitting and computes a per-student scale factor:

$$S = \text{clip}\left(\frac{t_{ed} \cdot W_{\text{frame}}}{e_{px} \cdot W_{\text{fitted}}} \cdot 100, S_{\min}, S_{\max}\right), \quad (5)$$

where e_{px} is the detected inter-eye distance in normalised coordinates, t_{ed} is the target eye distance as a fraction of frame width, and $[S_{\min}, S_{\max}]$ is a scale clamp derived from the frame’s aspect ratio (typically 125–145% for students, 120–145% for teachers). The final *offsetX* and *offsetY* are computed analytically so that the chin lands on the target position and the face is horizontally centred.

A related problem then came up: the pipeline was hardcoded to one particular InDesign template’s frame sizes. When LTID sent a different template with different frame dimensions, everything broke. That is when I proposed and built `ExportFrameDimensions.jsx` — a script that reads the template’s actual measurements and writes them to `frame_config.json`, so the Python code never needs to know what template it is working with.

3.7. InDesign Automation (ExtendScript)

I was asked on fairly short notice to automate the photo placement into Adobe InDesign. Since InDesign supports ExtendScript (JSX) for scripting, that became the path forward. I had no prior experience with ExtendScript or InDesign’s scripting API, so I had to learn the software in depth — understanding how spreads, layers, frames, and text threading work internally.

`ExportFrameDimensions.jsx` is run once on any new template. It switches the document to millimetre units, finds all portrait frames on the `Default /` layer, pulls out the group photo frame (largest by area), identifies the teacher frame (largest remaining), figures out the most common student frame size, and reads the horizontal red guide lines on the layer. From all of this it computes guide ratios and a scale clamp, then writes `frame_config.json`.

`AutoPlacePhotosAndNames.jsx` reads the `manifest.json` from Stages 3 and 4 and populates the template with zero manual intervention. Frames are collected from three layers (`Default, ,`), sorted into reading order with a 10 mm column-snap tolerance, and the group and teacher frames are separated out. For every student slot, it places the best-shot portrait, the ID-plate shot, and the student name. The face offsets from the manifest are applied through a `placeWithTransform` function that fills proportionally, centres, scales, and shifts.

Getting the name placement right was its own challenge. The template uses full-width underscores (U+FF3F) between adjacent kanji characters, styled in a specific colour (typically cyan). To preserve this, the script samples the underscore’s character style from the template *before* overwriting the text, writes the new name, and then re-applies the sampled style character by character. Absent students get a placeholder image while their name is still placed.

This script went through 13 versions before reaching its current state. Each version addressed a new edge case — threaded text frames, grouped objects on the spread, teacher frames that were inside groups, and so on. I am still iterating on it as new templates arrive from the client.

3.8. Error Handling

The pipeline maintains an error queue that catches two types of hard errors: duplicate attendance numbers (two different face clusters assigned the same number) and unknown student groups (no readable placard found for a cluster). Hard errors get excluded from the normal export and their images are copied to an `errors/` folder for manual review. A `error.resolutions.json` file lets an operator record corrections that persist across re-runs, so resolved errors do not reappear.

4. EXPERIMENTAL OBSERVATIONS

4.1. Setup

I ran pilot tests on data from multiple Japanese high schools during the internship, covering roughly 3,000 students in total. The reference dataset I use for detailed numbers in this report is Chihaya High School (), 3rd-year, classes 3-1, 3-3, and 3-5, with 99 portrait and teacher entries. All processing was done in Google Colab on a T4 GPU.

4.2. Identification Rate

Across the pilot data, the pipeline averages **2–3 manual reviews per 100 students**, which I tracked from the `review.queue.json` entries flagged after Stage 2. That puts the automatic identification rate at roughly 97–98%. The cases that still need human attention, in order of how often they come up, are: (i) motion-blurred or partially hidden placard digits, (ii) students who look similar enough (siblings, or just similar haircuts) that their embeddings fall within $\tau = 0.55$, (iii) placards held up too briefly to appear in any detected frame, and (iv) handwritten kanji on teacher cards that EasyOCR [2] misreads. A formal ground-truth evaluation against a manually labelled hold-out set is planned as part of the remaining internship work.

4.3. Face Alignment Stage

Table 2 shows the results from the Chihaya High School reference run. MediaPipe Face Mesh [8] produced valid landmarks for 86 out of 99 entries (86.9%). The remaining 13 (13.1%) got conservative fallback offsets, and there were zero complete failures.

The chin-y values cluster tightly around 0.49–0.50, and eye distances span 0.194 to 0.236 of the frame width. That narrow spread tells me the MediaPipe detections are consistent enough across different poses, expressions, and lighting to drive a uniform cropping policy for the whole class.

Table 2. Stage 4 face-alignment results, Chihaya High School

Metric	Value
Total entries (3 classes)	99
Offsets computed	86 (86.9%)
Fallback applied	13 (13.1%)
Complete failures	0 (0%)
Student frame	30 × 40 mm
Teacher frame	42 × 50 mm
Student scale clamp	125–145%
Teacher scale clamp	120–145%
Detected chin-y range	[0.478, 0.520]
Detected eye-distance range	[0.194, 0.236]
Per-image alignment runtime	~0.5 s

4.4. Processing Speed

Stage 2 (face detection, embedding, and OCR) takes about 1–3 seconds per image on the T4 GPU. Stage 4 (MediaPipe alignment) runs at roughly 0.5 seconds per image. For a typical school of around 300 students, end-to-end processing takes 15–30 minutes, with most of that time spent in Stage 2.

5. DISCUSSION

The face-embedding-first design proved itself during real pilot runs. At several schools, I noticed sequences where a student had turned to talk to someone, got photographed without a placard in frame for multiple shots, or had their placard reshot at a completely different point in the session. With an order-dependent matcher, each of these would have needed manual correction. Under the embedding-first approach, the clustering step absorbed them without any operator intervention. That validation gave me confidence that the design choice was right.

The threshold $\tau = 0.55$ sits comfortably below the typical same-person similarity in ArcFace embeddings (around 0.70 on LFW distributions [3]) and was the result of trial and error on our internal data.

The OCR migration from PaddleOCR [1] to EasyOCR [2] was driven by engineering constraints rather than accuracy. PaddleOCR *does* perform better on Japanese scene text [11], and that is exactly why I picked it originally. But its PaddlePaddle backend was heavy on GPU memory and kept clashing with InsightFace’s ONNX Runtime after Colab runtime restarts. EasyOCR, being lighter and built on a standard PyTorch + ONNX stack, dropped in cleanly. The multi-strategy teacher-card name extraction I built on top of it compensates for much of the accuracy gap.

The 13.1% MediaPipe fallback rate in the Chihaya run came mostly from portraits shot at steep angles or with partial occlusion. The fallback offsets ($offsetY \approx 78$ mm, $scaleFactor = 122\%$) are set to match the median of the computed values, so even fallback images end up in a visually acceptable position. Bringing this rate down with a secondary detector for off-frontal faces is on my to-do list.

Two limitations of the current clustering are worth mentioning. First, $\tau = 0.55$ is the same for every school regardless of class size; very large classes might eventually need an adaptive threshold or a graph-based algorithm like DBSCAN. Second, the running-centroid update could theoretically drift if an unusual pose enters a cluster early on, though I have not observed this in practice. A re-clustering

pass after Stage 2 would guard against it.

6. CONCLUSION AND FUTURE WORK

This internship at LTID, ongoing since late November 2025 with regular client feedback, has produced a working pipeline that has processed roughly 3,000 students across multiple schools. It currently flags 2–3 items per 100 students for manual review and produces InDesign-ready output through two ExtendScript automations that I wrote. My contributions include the face-embedding-first concept, the face grouping module, the package exporter, the MediaPipe face alignment module, and both JSX scripts (`ExportFrameDimensions.jsx` and `AutoPlacePhotosAndNames.jsx`). The roster parser and some OCR refinements came from team members during collaborative iteration.

What remains: (i) a formal ground-truth evaluation of the identification rate on a held-out labelled set with per-class confusion analysis; (ii) investigating an adaptive τ or switching to graph-based clustering; (iii) reducing the MediaPipe fallback rate with a secondary landmark detector for extreme poses; and (iv) packaging the pipeline as a standalone service so photographers do not need Colab. Final acceptance testing against LTID’s production yearbook output is the last milestone.

Note on tools. This report follows guidance on scientific writing from [12, 13, 14]. AI-assisted tools were used to help with code documentation and drafting; all design decisions, implementation work, and experimental results attributed to me are my own.

7. ARTIFACTS AND DEMONSTRATIONS

- **Code Repository:** https://github.com/CaramelMelon/LTID_Internship_Code
- **Video Presentation:** <https://youtu.be/UwCDJvTbLlg>
- **Pipeline Notebook:** `Colab_Photography_Pipeline.ipynb`
- **InDesign Scripts:** `AutoPlacePhotosAndNames_v13.jsx`, `ExportFrameDimensions.jsx`

8. REFERENCES

- [1] PaddlePaddle Authors, “PaddleOCR: Awesome multilingual OCR toolkits based on PaddlePaddle,” <https://github.com/PaddlePaddle/PaddleOCR>, 2020.
- [2] JaidevAI, “EasyOCR: Ready-to-use OCR with 80+ supported languages,” <https://github.com/JaidevAI/EasyOCR>, 2020.
- [3] Jiankang Deng, Jia Guo, Niannan Xue, and Stefanos Zafeiriou, “ArcFace: Additive angular margin loss for deep face recognition,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4690–4699.
- [4] Jiankang Deng, Jia Guo, Evangelos Ververas, Irene Kotsia, and Stefanos Zafeiriou, “RetinaFace: Single-shot multi-level face localisation in the wild,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 5203–5212.
- [5] Tereza Soukupová and Jan Čech, “Real-time eye blink detection using facial landmarks,” in *Proceedings of the 21st Computer Vision Winter Workshop (CVWW)*, 2016.
- [6] José Luis Pech-Pacheco, Gabriel Cristóbal, Jesús Chamorro-Martínez, and Joaquín Fernández-Valdivia, “Diatom autofocusing in brightfield microscopy: a comparative study,” in *Proceedings of the 15th International Conference on Pattern Recognition (ICPR)*, 2000, vol. 3, pp. 314–317.
- [7] Paul Viola and Michael Jones, “Rapid object detection using a boosted cascade of simple features,” in *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, 2001, vol. 1, pp. I–511–I–518.
- [8] Yury Kartynnik, Artsiom Ablavatski, Ivan Grishchenko, and Matthias Grundmann, “Real-time facial surface geometry from monocular video on mobile GPUs,” arXiv preprint arXiv:1907.06724, presented at the Third Workshop on Computer Vision for AR/VR at CVPR 2019, 2019.
- [9] OpenAI, “GPT-4 technical report,” arXiv preprint arXiv:2303.08774, 2023.
- [10] Gary Bradski, “The OpenCV library,” *Dr. Dobbs’s Journal of Software Tools*, vol. 25, no. 11, pp. 120–125, 2000.
- [11] IntuitionLabs, “Technical analysis of modern non-LLM OCR engines,” <https://intuitionlabs.ai/articles/non-llm-ocr-technologies>, 2026.
- [12] George D. Gopen and Judith A. Swan, “The science of scientific writing,” *American Scientist*, vol. 78, no. 6, pp. 550–558, 1990.
- [13] Simon Peyton Jones, “How to write a great research paper,” <https://www.microsoft.com/en-us/research/academic-program/write-great-research-paper/>, 2003.
- [14] Donald E. Knuth, “Literate programming,” *The Computer Journal*, vol. 27, no. 2, pp. 97–111, 1984.